IRC Results Snapshot

Refined GAD → IRC validation, status as of 2026-04-15

What This Note Covers

This note summarizes the *current* IRC validation workflow and the first scientifically meaningful refined rerun. It is intentionally compact: the goal is to preserve the key numbers, the threshold analysis, and a small amount of interpretation in one place.

Pipeline used here

1. Start from converged GAD trajectories from `noise_survey_300`.
2. These trajectories come from **dataset TS geometries** with Gaussian noise added to the `Transition1x` labeled transition-state coordinates.
3. Select a TS candidate from the saved trajectory (`best_nneg1`).
4. Recompute projected TS quality with HIP: n_{neg}, force norm, and $f_{\max}$.
5. If needed, refine the TS candidate with projected GAD.
6. Run IRC *from that TS* in both downhill directions using Sella's IRC routine.
7. Compare the two endpoints against the `Transition1x` labeled reactant/product using: aligned geometry (element-aware Kabsch/Hungarian RMSD) and bond-topology matching.

Important interpretation point The current labels `intended / half / unintended` mean: *does the HIP IRC from this GAD-found TS recover the dataset-labeled reactant/product pair?* This is a strong benchmark, but not a complete unsupervised analysis of all possible connected minima on HIP's PES.

Run Summary

The refined rerun analyzed here is the completed array `59367455_[0-2]`. It covered 30 converged TS candidates each at noise levels 0, 10, and 50 pm.

Noise	Total	IRC ran	Gate failed	Half	Topology-half only	Unintended
0 pm	30	12	18	7	1	4
10 pm	30	9	21	3	2	4
50 pm	30	4	26	0	0	4

Headline: the dominant failure mode is now `ts_quality_gate_failed`, not code failure. No intended rows were found in this rerun. The attempted cases become progressively worse with noise: mixed / half-intended at low noise, fully unintended by 50 pm.

Main Figure

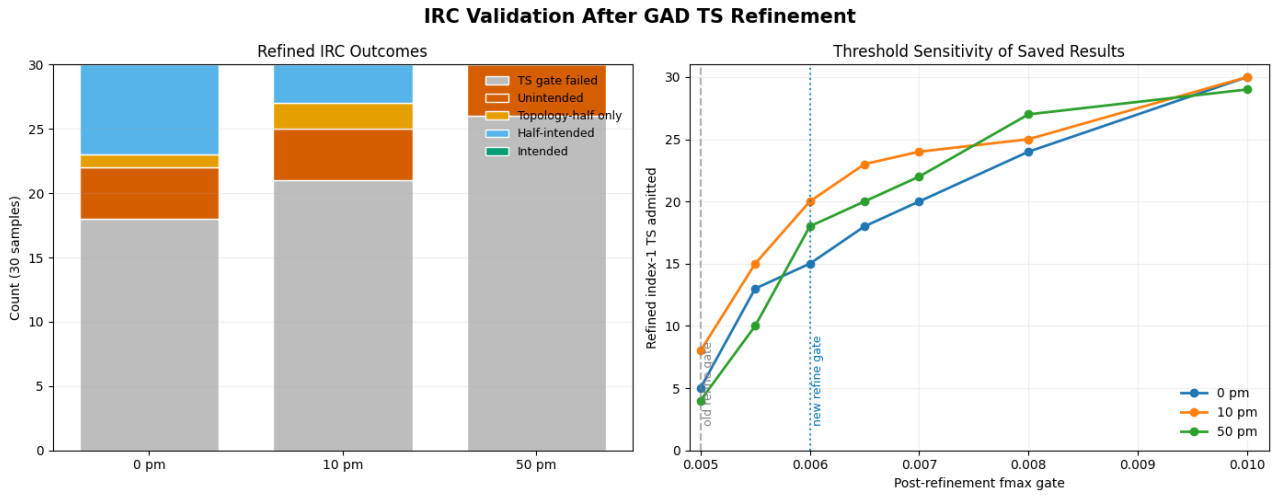

Figure 1: Left: outcome breakdown of the refined rerun. **Right:** number of refined index-1 candidates that would pass under looser post-refinement $f_{\max}$ gates, using the already-saved results.

Interpretation

- Noise hurts in two layers: fewer candidates survive TS screening, and the surviving ones connect less cleanly to the labeled endpoints.
- Even 0 pm is not "clean" under the stricter pipeline. This means the old convergence notion ($n_{\text{neg}} = 1$ plus loose force screening) was broad enough that some nominally converged points were not clean TSs by stricter $f_{\max}$-based standards.
- The threshold-sweep panel shows that the old post-refinement gate $f_{\max} < 0.005$ was too severe for the saved refined candidates.

Why The Force Gate Was Too Strict

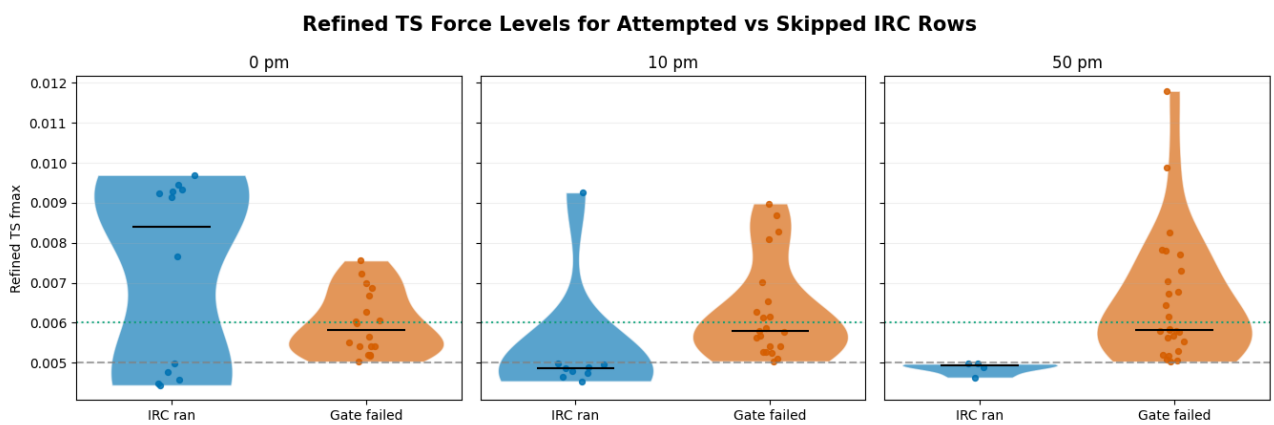

Figure 2: Refined TS $f_{\max}$ for rows that actually ran IRC versus rows that were skipped by the TS-quality gate. Dashed gray: old refined gate 0.005. Dotted green: new refined gate 0.006.

Most important observation: the previous policy was asymmetric.

- Unrefined candidates could go straight to IRC if they satisfied $f_{\max} < 0.01$.
- Refined candidates were only allowed through if they satisfied $f_{\max} < 0.005$.

That means some *improved* refined TSs were being rejected even though their $f_{\max}$ was smaller than already-accepted unrefined TSs.

Noise	Max accepted pre-gate $f_{\max}$	Min rejected refined $f_{\max}$	Median rejected refined $f_{\max}$
0 pm	0.009697	0.005027	0.005819
10 pm	0.009260	0.005033	0.005794
50 pm	—	0.005027	0.005824

Conclusion:

- 0.005 was stricter than necessary for a first refined IRC pass.
- The saved results support loosening to $f_{\max} < 0.006$, not all the way back to 0.01.
- Many borderline failures are clustered near 0.0052–0.0059, so 0.006 captures the obviously near-pass cases without becoming too loose.

Updated Rerun Settings

The new defaults already patched into the repo are:

- `--refine-steps 600` instead of 300
- `--refine-force-threshold 0.006` instead of 0.005

Why 600 steps? In the completed refined rerun, the refinement stage often hit the 300-step cap:

Noise	Rows hitting 300-step refinement cap	Mean refinement steps
0 pm	5	90
10 pm	7	174
50 pm	14	211

So the next rerun is not just "looser"; it also gives refinement more room to tighten genuinely borderline TSs.

What To Inspect First In 3D

The viewer bundles from the completed refined run are already available under: `/lustre07/scratch/memoozd/gadpl`

Recommended cases:

- `0 pm / sample 51`: refined strongly, still only half-intended
- `0 pm / samples 28, 33`: representative low-noise half-intended cases
- `10 pm / samples 175, 238`: useful geometry-vs-topology disagreement cases
- `50 pm / samples 85, 169`: clearly unintended high-noise examples

Each bundle contains:

- dataset reactant reference
- reverse IRC endpoint
- TS input

- forward IRC endpoint
- dataset product reference

So this is an endpoint/context view of the IRC outcome, not a dense movie of every internal IRC integration step.

Bottom Line

- The current refined IRC pipeline is now producing *real* scientific signal.
- The main issue is TS admissibility, not numerical IRC crashes.
- Noise degrades both TS quality and endpoint agreement.
- The old refined gate $f_{\max} < 0.005$ was too strict and internally inconsistent.
- The next rerun should be judged under the patched defaults: 600 refinement steps and post-refinement $f_{\max} < 0.006$.

What Is Running Right Now

Queue snapshot captured at 2026-04-15 16:16 UTC

JobID	Name	State	Time	Node / Reason
59385318_0	run_nr_then_gad_quic	R	8:40	ng20304
59385318_1	run_nr_then_gad_quic	R	8:40	ng20304
59385481_0	run_irc_validate.slu	R	8:40	ng20304
59385481_1	run_irc_validate.slu	R	8:40	ng20402
59385481_2	run_irc_validate.slu	R	8:40	ng20603
59385266_0	run_nr_then_gad.slur	R	12:09	ng20601
59385266_1	run_nr_then_gad.slur	R	12:09	ng20601
59385266_2	run_nr_then_gad.slur	R	12:09	ng20601
59385266_3	run_nr_then_gad.slur	R	12:09	ng30102
59385266_4	run_nr_then_gad.slur	R	12:09	ng30102
59385266_5	run_nr_then_gad.slur	R	12:09	ng30102
59385266_6	run_nr_then_gad.slur	R	12:09	ng30102
59385266_7	run_nr_then_gad.slur	R	12:09	ng20401
59385266_8	run_nr_then_gad.slur	R	12:09	ng20401
59385266_9	run_nr_then_gad.slur	R	12:09	ng20401
59385266_10	run_nr_then_gad.slur	R	8:40	ng30104
59385266_11	run_nr_then_gad.slur	R	8:40	ng20401

The important item for IRC is that the **new rerun is currently running as** `59385481_[0-2]`.